Natural Language Understanding in Task-Oriented Dialogue Systems

1. Problem Statement (Moga Patricia – Ciobanu Sergiu-Tudor)

This project is about building and testing the core parts of a task-oriented dialogue system that helps users find restaurants and get information. Unlike regular chatbots that just chat about anything, a task-oriented system is designed for a specific goal like helping someone find a place to eat, getting a phone number or even requesting an address.

The main Natural Language Processing (NLP) task is broken down into two main steps:

- **Natural Language Understanding (NLU):** This step turns normal human sentences into a structured format that a computer can easily understand, called Dialogue Acts. For example, if a user types *"I'm looking for a lebanese restaurant in the west part of town"*, the system needs to translate this into inform(food=lebanese, area=west, task=find). To handle this part, our project uses and compares two different methods: a traditional method using **Logistic Regression** and a transformer-based deep neural network approach using **TODBERT,** a pre-trained task-oriented dialogue BERT, which is specialized on intent recognition dialogue state tracking, slot filling etc.
- **Dialogue State Tracking (DST):** Human conversations happen step by step. People rarely say all their preferences in one single sentence, they usually give details over several turns (first the cuisine, then the area, and finally the price). The system needs a continuous memory to safely collect these details over time, while handling the natural mistakes and confusion that can happen when predicting text.

2. Proposed Solution

2.1 Theoretical Aspects

To make the system work smoothly, we do not try to guess everything at once. Instead, our solution uses a step-by-step pipeline divided into two main parts:

Rule-Based and Machine Learning Processing (NLU)

The system uses a smart mix of methods to guess what the user wants. At first, a simple rule-based script scans the text using exact keyword lists and patterns to catch clear words. To make the system flexible and smart, we then add two different machine learning options:

Approach A: The Statistical Logistic Regression Pipeline (Moga Patricia)

- **Multi-class Classifiers:** These are used for slots that can have multiple choices (like food, pricerange, and area). The model looks at the sentence and picks the best matching

option from a fixed list. A very important part here is the null class, which the model automatically assigns if the user did not mention that specific detail in the current turn.

- **Binary Classifiers:** These handle simple conversational actions that do not have extra details, like greetings (greet), saying goodbye (goodbye), saying yes (affirm), or asking for specific details like a phone number (request_phone). Each of these models makes a simple true-or-false decision to say if that specific action is happening or not.

Approach B: The Deep Learning Transformer Pipeline (TODBERT) (Ciobanu Sergiu-Tudor)

- The Transformer Approach reads the whole sentence in context and lets a single pre-trained model to do the classification. **TODBERT (TOD-DistilBERT-JNT-V1) is used in this scope**, a compact BERT that was already pre-trained on large amounts of task-oriented dialogue data, so it starts out "knowing" how people talk when booking restaurants, asking for phone numbers, or giving preferences.

- **Shared Contextual Encoder:** A single TODBERT encoder turns the user sentence into numerical values (a context vector). This representation understands word order and meaning, so paraphrases and unusual phrasings map to similar vectors.

- **Multi-class Value Heads:** On top of that shared vector sit small classifiers for the slots that can take many values (like food, pricerange, and area). Each one picks the best matching option from a fixed list, and includes a **null class** that is chosen automatically when the user did not mention that detail in the current turn.

- **Binary Action Heads:** A second group of small classifiers handles the simple actions that carry no value, such as greeting (greet), saying goodbye (bye), confirming (affirm), or asking for a specific detail (request_phone). Each makes a simple present-or-absent decision.

-

Probabilistic Dialogue State Tracking (DST) (Moga Patricia – Ciobanu Sergiu-Tudor)

To keep a safe memory of the conversation without getting confused by text errors, the tracker uses a probability distribution for each slot. Instead of just saving hard text strings, it updates percentages. At the very beginning of the chat, each slot starts with total uncertainty, meaning the None class has a 100% confidence score.

At the end of every conversational turn, the tracker updates its memory using a simple two-step rule:

1. **Fading the Past (Historical Decay):** The module checks how likely it is that the user did not mention anything new for a slot in the current turn (the None score). It then multiplies all previous memory scores by this factor, smoothly lowering the importance of old information.
2. **Adding the Present (Evidence Injection):** The new confidence scores from the current turn are added directly to the faded past scores.

To keep the memory clean and fast, the tracker automatically drops any prediction below a 0.01 confidence score and rounds the remaining numbers.

2.2 Data Set Used in Application – Description/Analysis (Ciobanu Sergiu-Tudor)

The application is built and tested using the **Dialog State Tracking Challenge 2 (DSTC2)** dataset, which is a popular benchmark focused entirely on the restaurant search domain. The dataset contains thousands of typed and annotated turns from real conversations between human users and an automated system.

The system focuses on tracking three main attributes (slots):

- food (like *italian*, *chinese*, *thai*, or *lebanese*)
- pricerange (like *cheap*, *moderate*, or *expensive*)
- area (like *center*, *north*, *south*, *east*, or *west*)

When looking closely at the data, we found two main challenges:

- **The Dominance of the Null Class:** Because people build up their preferences step by step, most slots are completely empty in any given sentence. This creates a huge imbalance where empty (null) labels are everywhere, and real words are rare. The models must be trained to handle this imbalance fairly.
- **Text Variation and Typos:** The data contains natural writing styles and mistakes, like switching between American and British spellings (*"center"* vs. *"centre"*) or quick typos (*"tailand"* instead of *"thai"*).

To make the project work, we split the dataset into two parts: a development set used to extract rules, clean text, and train models, and a separate test set used only for the final evaluation.

3. Implementation - Details

3.1 Pipeline Integration

The entire program is built as a modular pipeline where each part inherits from a shared base class. This design lets components plug into the system easily and process the conversation turn by turn. All information is shared through a central dialogue state object that works as a shared storage folder during the chat session.

3.2 Code Architecture and Execution Flow

To build both the traditional and the neural versions of our dialogue system, the project utilizes a few core Python libraries that handle everything from machine learning to configuration. The statistical pipeline relies heavily on scikit-learn to handle text vectorization and train the Logistic Regression models, while the neural pipeline uses the transformers library to load, fine-tune, and run the BERT model. Additionally, the system uses pyyaml to parse and read the configuration files where system parameters and thresholds are stored, and cloudpickle to safely serialize and save complex Python objects, internal states, and model definitions directly to disk.

3.2.1 Statistical Pipeline (Logistic Regression) (Moga Patricia)

The traditional machine learning implementation is organized into separate scripts that handle the system step by step:

- **The Preparation and Training Stage:** This script automatically reads and processes the dataset. It cleans the text, applies rule filters, and trains the machine learning models. Sentences are converted into word combinations (unigrams and bigrams) using count vectorizers. The models are trained using a balanced weight parameter so they do not get blinded by the massive amount of empty labels. The final trained models are saved into a single storage file (trained_models.pkl).
- **The Initialization Stage:** When the program starts up, each module loads its basic settings. The NLU module opens the saved model file to get ready for text and prepares its independent classification heads for the core categories.
- **The Turn Processing Stage:** Every time a user types a message, a chain reaction happens. The NLU module cleans the text (removes accents and fixes spellings), converts it into numerical features using the saved vectorizer, and predicts the current intents and slot choices along with their confidence scores.

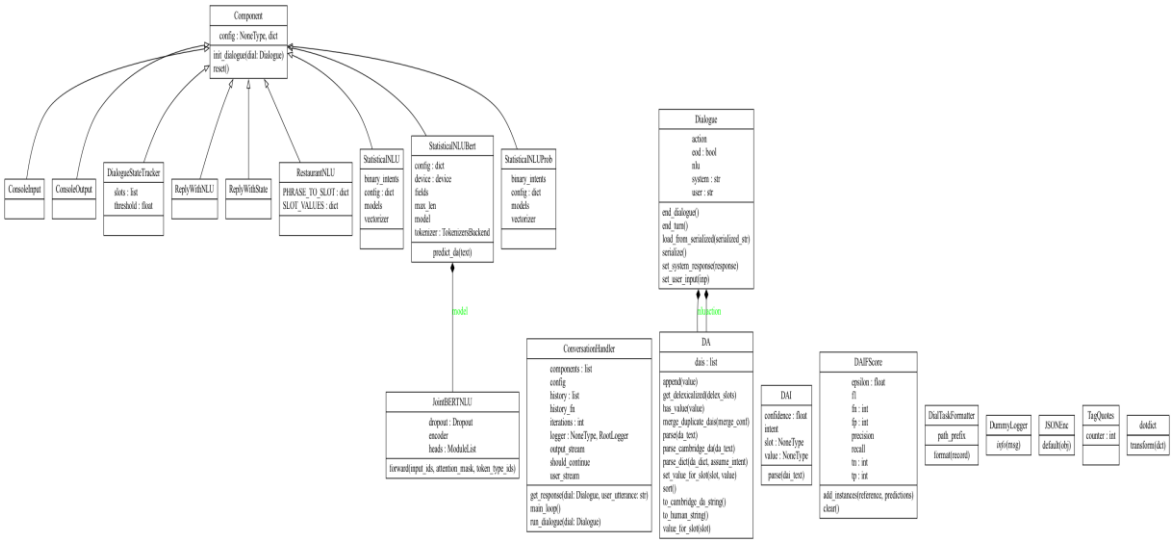
3.2.2 Neural Pipeline (BERT Integration) **(Ciobanu Sergiu-Tudor)**

The neural implementation follows the same step-by-step structure as the statistical one, but swaps the scikit-learn classifiers for a fine-tuned transformer.

- **The Preparation and Training Stage:** This script automatically reads and processes the development dataset. Each dialogue act is parsed with the framework's parse_cambridge_da method to recover the true intent-slot-value triples, and from these it automatically discovers every intent-slot pair to build one classification head per pair (multi-value heads get a null option, valueless pairs become binary heads). Sentences are converted into sub-word tokens by the TODBERT tokenizer, and the model is fine-tuned for a few passes over the data using the AdamW optimizer and a small learning rate. Because every head owns a null class, the model handles the massive amount of empty labels naturally.

- **The Turn Processing Stage:** Every time a user types a message, a chain reaction happens. The NLU module tokenizes the sentence, runs it once through the shared encoder to obtain the context vector, and then asks every head for its prediction in parallel. Any head that does not return null contributes a dialogue act item, and these are assembled into the final structured Dialogue Act for the current turn.

4. Dialogue System Application Pipeline **(Moga Patricia – Ciobanu Sergiu-Tudor)**



5. Experiments and Results

5.1 Evaluation Setup and Performance

To see how well the system works in a real scenario, we built a testing script that runs our components over the unseen test split of the DSTC2 data. The script reads the test inputs turn by turn, runs them through the pipeline, and saves all the predictions into a text file called predicted.txt.

This file writes out exactly one structured dialogue act per line, making it fully compatible with automated grading tools. The evaluation script checks everything at a turn-by-turn level, penalizing the application if it misses a preference, generates a false alarm, or confuses a restaurant detail.

5.2 Metrics and Observations

5.2.1 Logistic Regression Baseline Results **(Moga Patricia)**

We measured success using standard accuracy metrics evaluated across the test set. As shown in below, the statistical model achieved the following baseline results:

- **Precision: 0.989 (98.9%)**
- **Recall: 0.915 (91.5%)**
- **F1-Score: 0.951 (95.1%)**

Our experiments and these scores showed three main results:

- **High Precision and Core Reliability:** The high precision score of 98.9% indicates that when the model extracts an intent or a restaurant slot, it is almost always correct. This means the system rarely generates false alarms or misunderstands user constraints. The statistical models proved to be much more flexible than rigid keyword matching,

successfully understanding what the user wanted even when sentences were phrased in unique or unusual ways.

- **Balanced Recall Under Empty Data:** The recall score of 91.5% shows that the system captures the majority of user preferences across the turns. By using balanced training weights, the models did not become biased toward the dominant empty classes. The system remained highly reliable at picking up rare user requests and less common cuisines instead of simply ignoring them.
- **The Power of Cleaning Text:** The text-cleaning layer was incredibly helpful in achieving these solid percentages. Removing accents, standardizing spelling styles (like matching *"center"* to *"centre"*), and fixing common typing errors directly stopped word-mismatch bugs, keeping the vectorizer accurate and the classification scores stable across long chats.

5.2.2 BERT Results and Comparative Analysis **(Ciobanu Sergiu-Tudor)**

Evaluated on the same unseen test split with the same turn-by-turn scoring script, the TODBERT model reached near-perfect performance:

- **Precision: 0.998 (99.8%)**
- **Recall: 0.998 (99.8%)**
- **F1-Score: 0.998 (99.8%)**

Comparing the two approaches highlights three main results:

The biggest improvement over the Logistic Regression baseline is in recall, which increasesfrom 91.5% to 99.8%. Where the bag-of-words model occasionally missed a preference phrased in a rare or unusual way, the contextual encoder recognises the *meaning* of the sentence rather than just its exact words, so almost no user constraints slip through.

Precision also edged up from 98.9% to 99.8%, meaning the model very rarely raises a false alarm or invents a detail the user did not mention.

The combined F1-Score climbed from 95.1% to 99.8%. Because TODBERT works on sub-word tokens and was pre-trained on dialogue, it absorbs the natural text variation and typos in the data